# 为什么本项目采用 YOLO11 + LLM + Embedding，而不是直接使用视觉大模型

> 适用模块：`ai-engine`
> 目标场景：滑雪场事故视频分析、危险行为识别、定责建议生成

---

## 一、结论先行

本项目选择 **YOLO11 + 规则特征提取 + Embedding 检索 + LLM 生成** 的组合方案，而不是直接把整段视频交给视觉大模型（VLM）端到端处理，核心原因是：

1. **成本更可控**：视频逐帧送入视觉大模型，调用成本会随时长、分辨率、帧数快速上升；而 YOLO11 先把视频压缩成结构化轨迹和预警事件，再交给 LLM，只需要处理少量文本。
2. **性能更稳定**：YOLO11 在目标检测、跟踪、局部规则判断上延迟更低，适合实时或准实时场景；VLM 更适合复杂开放问答，不适合高频视频监控主链路。
3. **结果更可解释**：当前方案可以明确给出轨迹、速度、接近距离、逆行帧段、静止时长等中间证据，便于追责和复核；纯 VLM 方案往往“能回答”，但证据链较弱。
4. **后期微调性价比更高**：YOLO11、规则阈值、知识库、Prompt、Embedding 检索都可以独立优化；而直接微调视觉大模型成本高、周期长、数据要求大。
5. **工程解耦更强**：检测、规则、检索、生成各层可替换，便于后续渐进式升级，而不是把所有能力绑死在一个大模型 API 上。

一句话概括：

**视觉大模型适合做“高层语义理解补充”，但在本项目这种高频视频监控 + 结构化定责场景里，主链路更适合用 YOLO11 + LLM + Embedding 的分层架构。**

---

## 二、两种方案的定义

### 方案 A：当前方案

**YOLO11 + 行为规则引擎 + Embedding 检索 + LLM 生成**

处理流程：

```
视频
  ↓
YOLO11 检测/跟踪
  ↓
轨迹点、速度、距离、逆行/超速/碰撞风险等结构化事件
  ↓
Embedding 检索相关规范与案例
  ↓
LLM 基于结构化证据 + 检索上下文生成定责建议
```

### 方案 B：直接视觉大模型

**整段视频/抽帧图片 → 视觉大模型 → 直接输出事故分析与定责建议**

处理流程：

```
视频 / 抽帧图像
  ↓
视觉大模型（VLM）
  ↓
直接输出行为判断、责任划分、建议措施
```

---

# 三、从调用成本角度对比

## 3.1 为什么视觉大模型成本更容易失控

视觉大模型的计费通常与以下因素强相关：

- 上传图片数量
- 视频时长
- 抽帧频率
- 输入分辨率
- 多轮推理次数
- 输出 token 数量

如果一个 30 秒视频按 1fps 抽帧，就是 30 张图；如果按 3fps 抽帧，就是 90 张图。若一场雪场高峰期同时有多路摄像头，每路都要持续分析，调用量会非常快地放大。

而本项目当前方案中：

- YOLO11 在本地或本服务内部推理
- 行为检测主要是数值计算和规则判断
- 送给 LLM 的只是结构化文本摘要
- Embedding 只在知识库建索引或查询时调用

因此，LLM 侧的输入规模通常只与以下内容有关：

- 预警条目数量
- 轨迹摘要数量
- 检索到的知识片段数量

这相比“几十张图甚至整段视频直接送模型”要便宜得多，也更可预测。

## 3.2 成本结构对比

| 维度 | YOLO11 + LLM + Embedding | 直接视觉大模型 |
|---|---|---|
| 视频主处理成本 | 主要是本地 GPU / CPU 推理成本 | 主要是 API 调用成本 |
| 单次输入规模 | 结构化文本，通常较小 | 多张图或整段视频，通常较大 |

| 维度 | YOLO11 + LLM + Embedding | 直接视觉大模型 |
| --- | --- | --- |
| 高并发时成本 | 增长相对平缓 | 增长明显，且与帧数强相关 |
| 成本可预估性 | 高 | 中到低 |
| 是否适合长时间监控 | 更适合 | 通常不划算 |

## 3.3 本项目场景下的实际含义

滑雪场监控不是“一次性问答”，而是：

- 多摄像头
- 长时运行
- 高峰期可能并发分析
- 部分任务只是筛查，不一定都需要最终定责

如果全部主流程都走视觉大模型，相当于把最贵的能力用于每次基础检测，性价比不高。

当前方案则可以做到：

- 绝大多数时间由 YOLO11 完成基础检测
- 只有检测到预警时才触发定责生成
- 只有生成定责时才调用 LLM
- 只有做知识检索时才用 Embedding

这是一种“把昂贵能力后置”的架构，更适合长期运行。

---

# 四、从性能与实时性角度对比

## 4.1 YOLO11 在主链路上更适合实时检测

YOLO11 的强项是：

- 目标检测
- 多目标跟踪配合
- 边缘部署或本地部署
- 推理延迟低
- 吞吐高

对于滑雪场事故分析，第一步最重要的是先回答这些问题：

- 画面里有几个人？
- 每个人的轨迹是什么？
- 是否逆行？
- 是否超速？
- 是否存在短时间接近导致碰撞风险？
- 是否长期静止疑似摔倒？

这些问题本质上都适合通过检测 + 跟踪 + 数值规则完成，而不是交给大模型做“理解式推理”。

## 4.2 视觉大模型更像高层分析器，不像实时检测器

视觉大模型擅长的是：

- 场景描述
- 多模态问答
- 复杂语义理解
- 开放式解释

但它不擅长：

- 高帧率连续处理
- 稳定输出精确轨迹
- 长时间视频流低成本监测
- 在每帧级别上给出稳定一致的量化指标

换句话说，VLM 更适合作为“专家会诊”，不适合作为“前线巡逻员”。

## 4.3 延迟链路对比

### 当前方案

```
视频 → YOLO11 → 轨迹/事件 → LLM 生成结论
```

优势：

- 大部分耗时在视觉检测，但这一步可本地优化
- 后续传给 LLM 的信息量小
- 可先出结构化告警，再补充生成定责文案

### 直接视觉大模型方案

```
视频/多帧图 → 上传到 VLM → 模型理解 → 输出结论
```

劣势：

- 上传耗时明显
- 模型处理图像的时延高于纯文本
- 长视频必须抽帧，否则根本无法高效处理
- 推理延迟和网络波动耦合更强

因此，从工程性能上看，**YOLO11 做前置筛查，LLM 做后置总结**，更符合实时系统设计。

---

# 五、从准确性与稳定性角度对比

## 5.1 定责不是纯感知问题，而是“感知 + 规则 + 证据 + 法规”问题

滑雪事故责任认定并不是简单识别“有没有人摔倒”，而是要综合：

- 轨迹方向
- 速度变化
- 双方相对位置
- 接近过程
- 是否处于超越关系
- 安全规范条文
- 历史案例相似性

也就是说，这个任务并不是纯视觉分类，而是一个复合决策问题。

当前方案把这个任务拆成三层：

1. **YOLO11** 负责感知世界
2. **规则/特征层** 负责把感知结果转成可计算证据
3. **Embedding + LLM** 负责把证据映射到规范与案例，并生成自然语言结论

这种拆层让系统更稳定，因为每一层都只做自己擅长的事。

## 5.2 视觉大模型直接定责的风险

如果直接让视觉大模型看图后输出“谁负主要责任”，会面临几个问题：

- 责任判断依据不稳定
- 同一视频不同抽帧方式可能影响结论
- 对细小位移、连续帧运动趋势的量化能力不一定稳定
- 很难精准复现“为什么判 70% / 30%”
- 在涉及争议场景时，模型可能给出合理但不严谨的解释

而本项目需要的是可落地、可展示、可追溯的系统，不只是“看起来会回答”。

---

# 六、从可解释性与合规展示角度对比

在事故定责场景里，系统输出不仅要“像”，还要“能解释”。

## 当前方案的可解释证据包括：

- 轨迹 ID
- 每帧坐标
- 总位移
- 逆行连续帧数
- 超速峰值
- 碰撞接近帧数
- 静止持续时长
- 命中的规范条文
- 命中的相似案例片段

这样输出的结论可以追问：

- 为什么说逆行？
- 触发阈值是多少？
- 为什么说存在碰撞风险？
- 依据的规则条文是哪条？
- 参考了什么历史案例？

而直接 VLM 方案通常较难给出完整、稳定、结构化的证据链。

对于比赛答辩、产品汇报、后续商业化试点来说，这一点非常重要。

---

# 七、从后期微调与优化性价比角度对比

## 7.1 当前方案可以局部优化，成本低

当前架构支持多层次低成本优化：

### 可优化层 1：YOLO11 检测模型

可以通过补充滑雪场数据，微调：

- 滑雪者检测框稳定性
- 遮挡场景鲁棒性
- 摔倒姿态识别能力
- 摄像头角度迁移能力

这类微调通常比端到端训练视觉大模型更现实。

### 可优化层 2：规则与阈值

可以根据雪道、分辨率、帧率、镜头俯仰角，调整：

- 逆行判定阈值
- 超速阈值
- 接近距离阈值
- 静止时长阈值

这是最便宜、最快见效的优化手段。

### 可优化层 3：Embedding 检索层

可以逐步优化：

- 知识库文档质量
- 文本切块策略
- 检索 top_k
- Embedding 模型
- 召回与重排策略

### 可优化层 4：LLM 生成层

可以优化：

- Prompt 模板
- 输出结构
- 责任比例表达方式
- 是否要求引用条文
- 是否要求输出风险建议

这类优化通常无需重新训练大模型，只需调 Prompt 和知识库。

## 7.2 直接视觉大模型的微调门槛更高

如果采用直接 VLM 路线，后续要提升效果，往往会遇到：

- 需要大量高质量视频标注数据
- 微调算力成本高
- 训练周期长
- 多模态对齐复杂
- 评估口径难统一
- 模型版本升级后兼容性不稳定

而且即便微调完成，系统仍然可能缺乏足够清晰的中间证据，不利于业务落地。

## 7.3 性价比结论

如果目标是“做一个可答辩、可部署、可迭代、可控成本的系统”，那么：

- **YOLO11 + LLM + Embedding**：更适合中小团队和 MVP 到产品化过渡
- **直接视觉大模型**：更适合预算充足、数据充足、且问题更偏开放语义理解的团队

---

# 八、从系统工程与可维护性角度对比

## 8.1 当前方案模块边界清晰

当前方案天然分层：

- YOLO11：负责“看见”
- 行为检测器：负责“算出风险”
- Embedding 检索：负责“找依据”
- LLM：负责“写结论”

每层都可以单测、替换、调优、降级。

例如：

- LLM 服务不可用时，可回退到规则引擎
- Embedding 模型可以替换，不影响 YOLO11
- YOLO11 可以升级，不影响 RAG 结构

- 规则引擎可以先行落地，不阻塞 LLM 接入

## 8.2 直接 VLM 方案耦合更强

如果一个模型同时承担：

- 视觉理解
- 行为识别
- 责任分析
- 条文引用
- 建议生成

那它一旦不稳定，整条链路都会受影响。并且你很难定位问题到底出在：

- 抽帧问题
- 视觉理解问题
- 时序理解问题
- 责任推理问题
- 法规引用问题

从维护角度看，分层系统明显更优。

---

# 九、一个更贴近业务的比喻

可以把两种方案理解成：

## 当前方案

- YOLO11 是现场交警和测速仪
- 规则引擎是事故判定流程表
- Embedding 是法规案例检索员
- LLM 是最终出具说明书的文员

每个人只做自己最擅长的事，所以整体效率高、成本低、责任清晰。

## 直接视觉大模型方案

- 相当于把“现场勘查、测量、查法规、写结论”全部交给一个全能专家

理论上很强，但现实中：

- 出场费高
- 排队慢
- 每次都请他看完整段视频不划算
- 还不一定愿意给你完整的量化证据链

这就是为什么在本项目里，不适合让视觉大模型直接担任主链路。

---

# 十、适合引入视觉大模型的地方

这并不意味着视觉大模型完全没用。相反，它更适合作为当前架构的增强层，而不是替代层。

更合适的接入方式包括：

1. **对高风险片段做二次复核**
    - 只有 YOLO11 检测到疑似事故后，才截取关键帧交给 VLM
    - 用于补充场景描述、姿态解释、环境语义
2. **处理规则难覆盖的复杂场景**
    - 例如多人遮挡
    - 例如异常姿态但非标准跌倒
    - 例如雪具散落、护栏碰撞等长尾情况
3. **用于生成更自然的解释性文本**
    - 把结构化检测结果和关键帧一起输入 VLM
    - 输出更贴近人工事故报告的话术
4. **作为离线质检工具**
    - 用于抽查 YOLO11 结果质量
    - 用于辅助构造标注集和困难样本集

也就是说，**VLM 更适合当“增强插件”，而不是“主发动机”。**

---

# 十一、最终结论

综合调用成本、性能、可解释性、后期微调性价比、工程可维护性几个维度，本项目采用 **YOLO11 + LLM + Embedding** 的方案更合理。

## 选择这套方案的核心原因可以概括为：

- **更省钱**：把最贵的大模型调用放在最后一步，只在必要时触发
- **更快**：检测主链路走 YOLO11，适合实时处理
- **更稳**：感知、规则、检索、生成分层，各司其职
- **更能解释**：能给出轨迹、阈值、案例、条文等证据链
- **更好迭代**：规则、模型、知识库、Prompt 都能独立优化
- **更适合产品化**：支持降级、替换、扩展和长期运行

因此，对于“滑雪场事故监测与智能定责”这样的业务场景，当前方案不是保守，而是更符合工程现实和商业落地逻辑的选择。